

World Cup Sticker Manager

Gestor de Cromos Mundialistas

Proyecto Final — Módulo Backend Pro

Aplicación web ASP.NET Core MVC para administrar y explorar un catálogo de cromos del mundial: países, equipos, jugadores, cromos y álbumes.

Integrantes	Nicolás López Cadena
	(completar)
Tecnología	ASP.NET Core MVC (.NET 10) + EF Core 10
Base de datos	SQL Server Express
Fecha	Junio 2026

1. Objetivo del proyecto

El objetivo de este proyecto es desarrollar una aplicación web completa que permita gestionar una colección de cromos (stickers) del Mundial de Fútbol. El sistema modela el dominio real de un álbum de cromos: cada **país** agrupa **equipos** (selecciones), cada equipo reúne a sus **jugadores**, y cada **cromo** representa a un jugador dentro de un equipo y pertenece a un **álbum**. Además se modela la relación muchos-a-muchos entre **usuarios** y los cromos que poseen.

La aplicación implementa un **CRUD completo** sobre las cinco entidades principales, navegación entre entidades relacionadas, filtros de búsqueda, subida de imágenes, validaciones de cliente y servidor, un **dashboard** con indicadores y una interfaz responsiva con la identidad visual del Mundial.

Objetivos específicos

- Aplicar el patrón **MVC** con separación de responsabilidades (Modelos, Vistas, Controladores, Servicios y ViewModels).
- Modelar relaciones 1:N y N:M con **Entity Framework Core** y Fluent API.
- Implementar validaciones robustas con **Data Annotations** a nivel de cliente y servidor.
- Construir una experiencia de usuario moderna y coherente con **Bootstrap 5**.
- Sembrar datos iniciales de forma automática para una demostración inmediata.

2. Tecnologías utilizadas

Capa	Tecnología
Lenguaje	C#
Framework	ASP.NET Core MVC (.NET 10)
ORM	Entity Framework Core 10
Base de datos	SQL Server Express
Frontend	Razor Views + Bootstrap 5 + Bootstrap Icons
Patrón	MVC + Servicios + ViewModels
Herramientas	Navicat / SSMS, Visual Studio / dotnet CLI

3. Modelo de base de datos

Entidades y relaciones

- Pais (1) —< Equipo (N)
- Equipo (1) —< Jugador (N)
- Equipo (1) —< Cromo (N)
- Jugador (1) —< Cromo (N)
- Album (1) —< Cromo (N)
- Usuario (N) >—< Cromo (N) — tabla intermedia **UsuarioCromo** (con *FechaAdquisicion* y *Estado*).

Las claves foráneas usan **DeleteBehavior.Restrict** para impedir borrados que dejen registros huérfanos; la relación con **Album** y la tabla **UsuarioCromo** usan borrado en cascada. La tabla **UsuarioCromo** tiene una **clave compuesta** (Usuariold + Cromold).

Índices únicos

- **Cromo.NumeroCromo** — cada cromo tiene un número irrepetible.
- **Pais.CodigoFifa** — código FIFA de 3 letras único por país.
- **Usuario.Email** — correo único por usuario.

Datos sembrados (seed automático)

Entidad	Cantidad
Países	8
Equipos	8
Jugadores	20
Cromos	20
Álbumes	1
Usuarios	2
UsuarioCromo	4

4. Capturas del sistema

4.1 Dashboard / Página de inicio

Panel principal con indicadores (totales por entidad), últimos cromos agregados y equipos destacados.

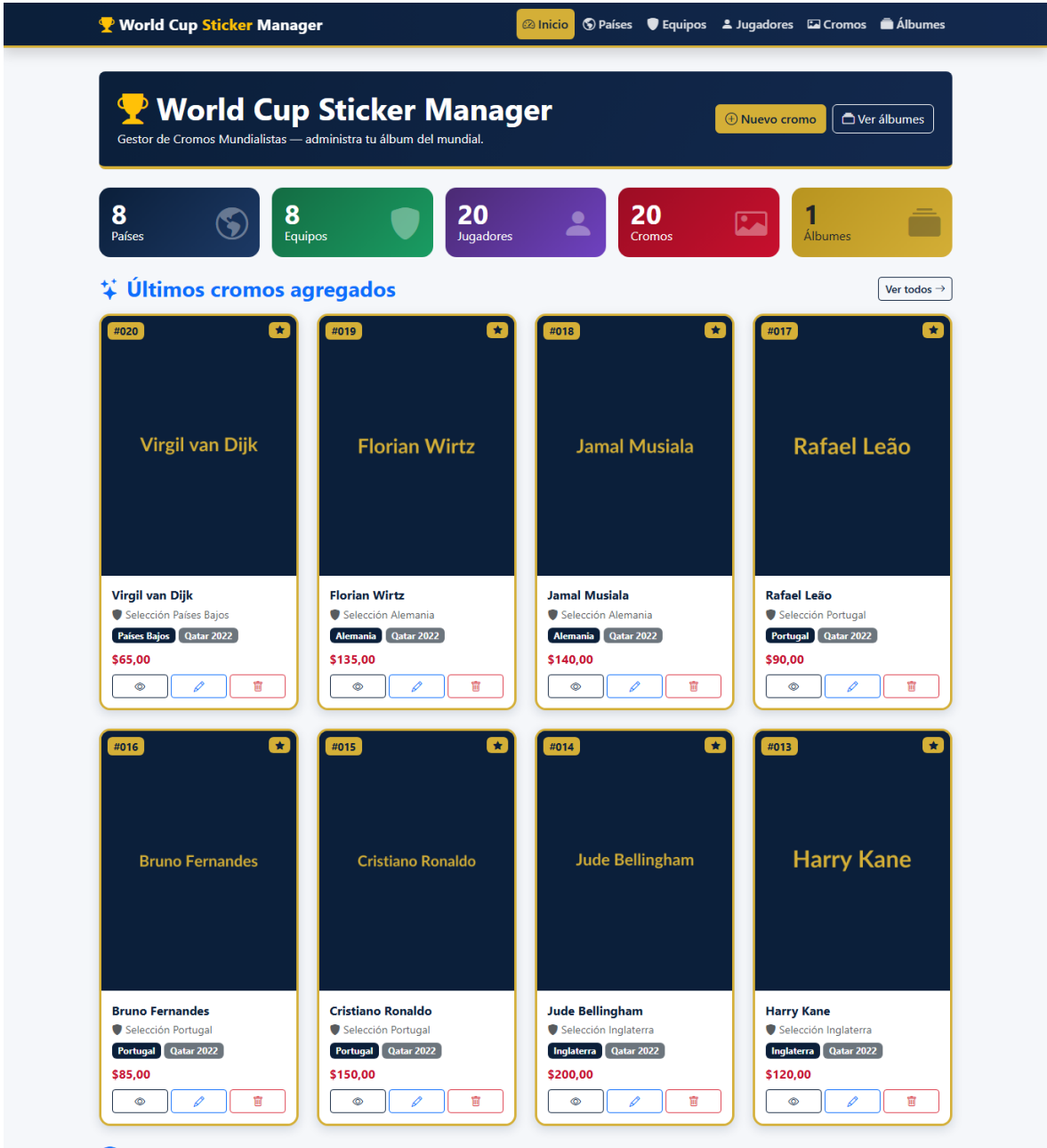


Figura 1. Dashboard con tarjetas de indicadores y últimos cromos.

4.2 Catálogo de cromos

Vista de catálogo tipo carta coleccionable con filtros por número, jugador, equipo, país, álbum y edición.

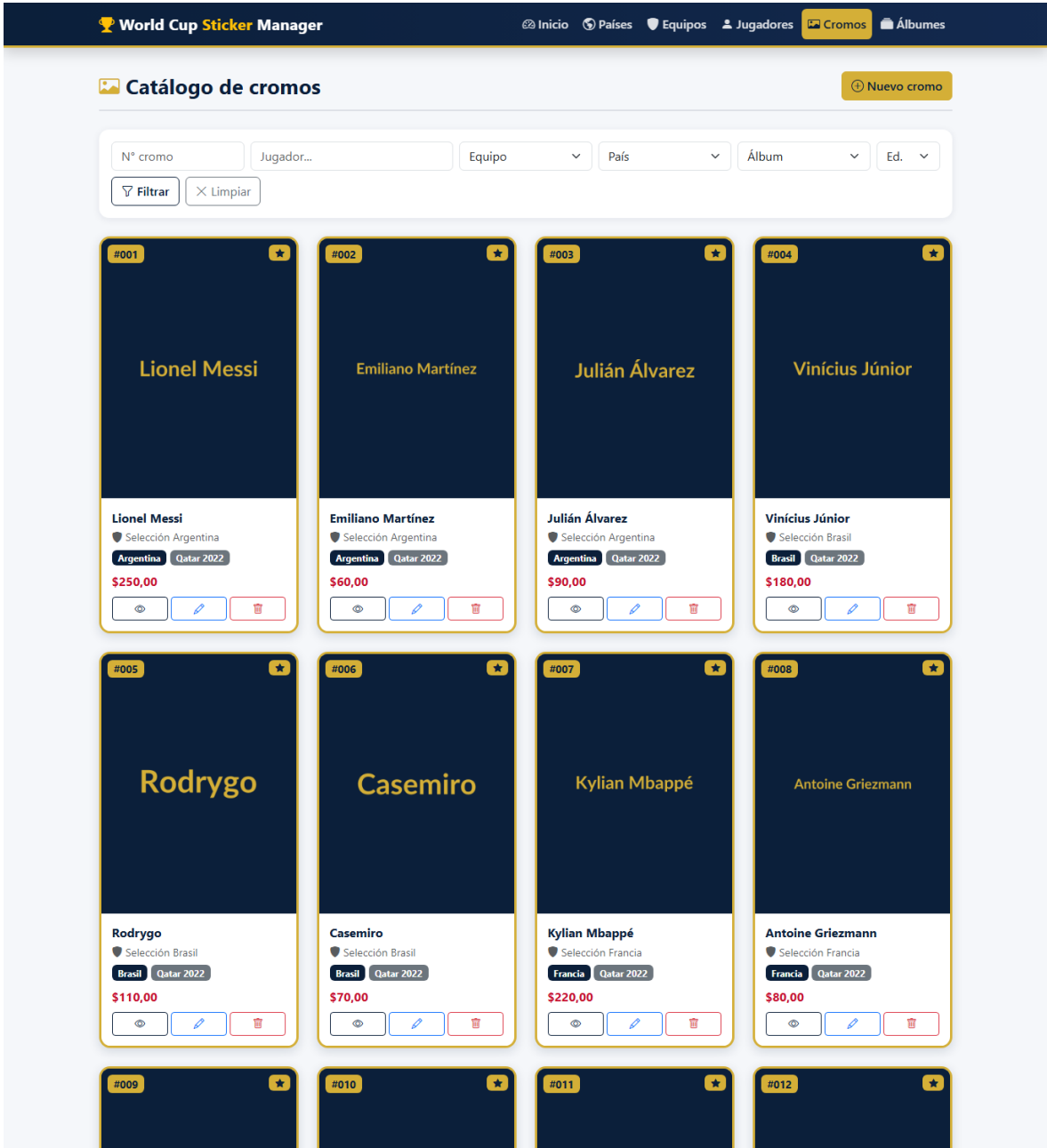


Figura 2. Catálogo de cromos con filtros y tarjetas coleccionables.

4.3 Detalle de álbum

Detalle del álbum con barra de progreso (cromos registrados sobre la meta) y el listado de cromos que lo componen.

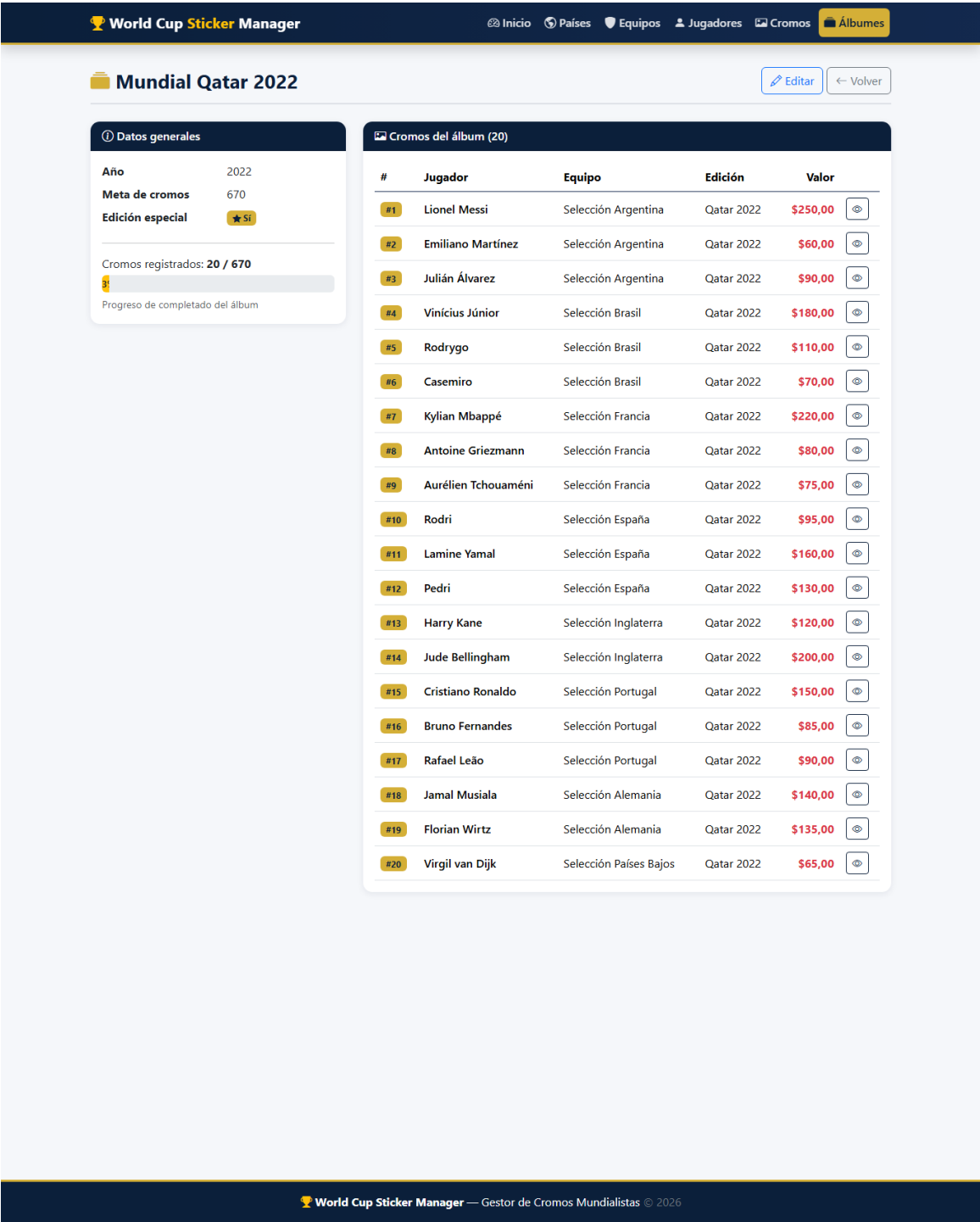


Figura 3. Detalle de álbum con progreso de completado.

4.4 Listado de jugadores

Listado de jugadores con filtros por nombre, país, equipo y posición.

World Cup Sticker Manager

Inicio

Países

Equipos

Jugadores

Cromos

Álbumes

Jugadores

Nuevo jugador

Nombre...

Todos los países

Todos los equipos

Todas las posiciones

Filtrar

X Limpiar

#	Nombre	Posición	Equipo	País	Cromos	Acciones
7	Antoine Griezmann	Mediocampista	Selección Francia	Francia	1	  
8	Aurélien Tchouaméni	Mediocampista	Selección Francia	Francia	1	  
8	Bruno Fernandes	Mediocampista	Selección Portugal	Portugal	1	  
5	Casemiro	Mediocampista	Selección Brasil	Brasil	1	  
7	Cristiano Ronaldo	Delantero	Selección Portugal	Portugal	1	  
23	Emiliano Martínez	Portero	Selección Argentina	Argentina	1	  
17	Florian Wirtz	Mediocampista	Selección Alemania	Alemania	1	  
9	Harry Kane	Delantero	Selección Inglaterra	Inglaterra	1	  
10	Jamal Musiala	Mediocampista	Selección Alemania	Alemania	1	  
10	Jude Bellingham	Mediocampista	Selección Inglaterra	Inglaterra	1	  
9	Julián Álvarez	Delantero	Selección Argentina	Argentina	1	  
10	Kylian Mbappé	Delantero	Selección Francia	Francia	1	  
19	Lamine Yamal	Extremo	Selección España	España	1	  
10	Lionel Messi	Delantero	Selección Argentina	Argentina	1	  
8	Pedri	Mediocampista	Selección España	España	1	  
17	Rafael Leão	Extremo	Selección Portugal	Portugal	1	  
16	Rodri	Mediocampista	Selección España	España	1	  
10	Rodrygo	Extremo	Selección Brasil	Brasil	1	  
7	Vinícius Júnior	Extremo	Selección Brasil	Brasil	1	  
4	Virgil van Dijk	Defensa	Selección Países Bajos	Países Bajos	1	  

20 jugador(es) encontrado(s).

Figura 4. Listado de jugadores con filtros de búsqueda.

5. Decisiones técnicas

Arquitectura por capas

Se siguió el patrón **MVC** extendido con una capa de **Servicios** (*FileUploadService*) y **ViewModels** para el dashboard y los filtros. Los controladores son delgados: orquestan el acceso a datos vía *ApplicationDbContext* y delegan responsabilidades transversales (subida de archivos) a servicios inyectados con DI.

Migraciones + Seed automático

Al arrancar, la aplicación aplica las migraciones pendientes (*MigrateAsync*) y ejecuta *DbInitializer*, que siembra los datos solo si la base está vacía. Esto permite levantar el proyecto y verlo funcionando sin pasos manuales de carga.

8 países y 8 selecciones

Aunque el mínimo era de 5 países, se modelaron **8 países con su respectiva selección nacional** para dar coherencia temática al dominio mundialista, manteniendo una relación 1:1 natural entre país y selección.

Manejo de imágenes

El sistema admite **dos modos**: subir un archivo (que se guarda en *wwwroot/images* con un nombre único basado en *Guid*) o indicar una **URL** externa. El seed utiliza URLs públicas para banderas y placeholders, de modo que la demostración no depende de archivos locales.

Enum como texto

El estado del cromó (Nuevo, Repetido, Intercambiable, Favorito) se persiste como **cadena** (*HasConversion<string>*) para que los datos sean legibles directamente desde Navicat / SSMS sin necesidad de traducir valores numéricos.

6. Seguridad y validaciones

- Validación de **extensión** (.jpg, .jpeg, .png, .webp) y **tamaño** (máx. 2 MB) de las imágenes subidas.
- Nombres de archivo generados con **Guid**: nunca se usa el nombre original del archivo del usuario.
- Imágenes guardadas únicamente dentro de **wwwroot/images**.
- **ValidateAntiForgeryToken** en todos los formularios POST (protección CSRF).
- Validaciones de dominio con **Data Annotations** (rangos, longitudes, expresiones regulares, campos requeridos) en cliente y servidor.
- Confirmación antes de eliminar y **bloqueo de borrado** cuando existen registros relacionados, evitando inconsistencias.

7. Instrucciones de ejecución

Requisitos

- .NET 10 SDK
- SQL Server Express (o LocalDB)
- (Opcional) Navicat o SSMS para inspeccionar la base de datos

Pasos

1) Configurar la cadena de conexión en **appsettings.json**:

```
Server=.\SQLEXPRESS;Database=WorldCupStickersDb;  
Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True
```

2) Restaurar, crear la base de datos y ejecutar:

```
dotnet restore  
dotnet ef database update  
dotnet run
```

La aplicación aplica las migraciones y siembra los datos iniciales automáticamente. Luego se abre el navegador en la URL indicada por la consola (por ejemplo *http://localhost:5199*).

8. Integrantes y repositorio

Integrantes

- Nicolás López Cadena
- *(completar con el resto de integrantes)*

Repositorio

(completar con el enlace de GitHub del proyecto)